Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

Thème principal : Algorithmique - Structures itératives (POUR, TANT QUE, RÉPÉTER), Variant et Invariant de boucle

Synthèse exécutive : Ce cours pose les fondements algorithmiques de la programmation structurée à travers l'itération. Répéter des instructions sans dupliquer le code est indispensable pour traiter des volumes de données variables de manière dynamique et maintenable. L'enjeu majeur est double : savoir choisir la structure de boucle adaptée à la situation (bornée avec POUR vs non bornée avec TANT QUE ou RÉPÉTER) et appliquer la rigueur mathématique pour garantir la qualité du code. On y apprend à prouver scientifiquement qu'un algorithme s'arrête (preuve de terminaison via le 'variant de boucle') et qu'il produit le résultat exact attendu (preuve de correction via 'l'invariant de boucle').

Typologie et Choix des Structures Itératives

En algorithmique, une structure itérative permet d'exécuter un bloc d'instructions un certain nombre de fois sans dupliquer le texte du programme. Choisir la bonne boucle repose sur deux critères clairs : 1. Connaît-on le nombre d'itérations à l'avance ? Si oui, on privilégie la boucle 'POUR'. Elle prend en charge la gestion du compteur (initialisation, incrémentation et test de fin). 2. Si le nombre de tours dépend d'une condition dynamique (ex: une donnée saisie par l'utilisateur ou la lecture d'un fichier), on utilise une boucle non bornée. Si le traitement peut ne jamais être exécuté selon la situation initiale, la boucle 'TANT QUE' s'impose (0 à N exécutations). Si le traitement doit impérativement s'exécuter au moins une fois avant de tester la condition (ex: un menu de choix ou un contrôle de saisie), on utilise la boucle 'RÉPÉTER... JUSQU'À' (1 à N exécutations).

- Anti-pattern du copier-coller : Erreur de conception majeure rendant le code rigide, illisible et incapable de gérer un nombre variable de données.
- Itération Bornée (POUR) : Utilisée lorsque le nombre exact d'itérations est fixe et connu au préalable. La variable compteur progresse automatiquement entre une borne inférieure et supérieure.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition logique est évaluée AVANT le corps de la boucle. Si elle est fausse d'entrée, le bloc s'exécute 0 fois.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le bloc d'instructions est exécuté D'ABORD, puis la condition d'arrêt est évaluée. Garantit au moins 1 exécution.

Cas concret enseignant : *L'analogie du coureur sur piste d'athlétisme : Si l'entraîneur ordonne 'Fais 10 tours de piste', c'est une boucle POUR. S'il ordonne 'Cours tant qu'il ne pleut pas', c'est un TANT QUE : s'il pleut déjà au vestiaire, le coureur ne commence même pas (0 tour). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À car il est obligé de faire au moins le premier tour avant d'évaluer son niveau de fatigue.*

Preuve Formelle de Terminaison : Le Variant de Boucle

Pour démontrer rigoureusement qu'un algorithme ne va pas 'planter' ou boucler à l'infini, l'informaticien doit formaliser un variant de boucle. Mathématiquement, la preuve s'appuie sur le principe du bon ordre des entiers naturels : il est impossible de construire une suite d'entiers naturels qui soit à la fois infinie, strictement décroissante et minorée par zéro. Ainsi, si l'on identifie une expression entière V fonction de nos variables qui : 1) décroît strictement à chaque passage dans la boucle, et 2) garantit l'arrêt dès qu'elle atteint une valeur minimale (souvent 0), alors la boucle s'arrêtera obligatoirement en un nombre fini d'étapes.

- Boucle Infinie : Dysfonctionnement critique bloquant le programme lorsque la condition d'arrêt n'est jamais atteinte.
- Variant de Boucle : Grandeur entière calculée à partir des variables du programme qui mesure la 'distance' restant à parcourir avant l'arrêt.

• Décroissance Stricte : Propriété imposant au variant de diminuer d'au moins 1 entier à chaque tour de boucle ($V_{k+1} < V_k$).

• Minoration / Borne Inférieure : Limite plancher (souvent 0) sous laquelle le variant ne peut pas descendre sans que la boucle ne s'arrête.

Cas concret enseignant : *La métaphore du réservoir d'essence d'une voiture : À chaque kilomètre parcouru (une itération), le volume d'essence dans le réservoir (le variant) diminue obligatoirement d'une quantité stricte. Comme le réservoir ne peut pas descendre en dessous de 0 litre (le minorant), la voiture finira obligatoirement par s'arrêter. Si la jauge ne baissait pas à chaque kilomètre, la voiture roulerait à l'infini : c'est l'équivalent exact du bug de la boucle infinie.*

Preuve Formelle de Correction : L'Invariant de Boucle

Savoir qu'une boucle s'arrête est indispensable, mais garantir qu'elle calcule la bonne valeur l'est tout autant. L'invariant de boucle est l'équivalent du raisonnement par récurrence appliqué aux algorithmes. On établit une propriété logique P reliant les variables du programme. On montre : 1) que P est vraie à l'état initial (Initialisation) ; 2) que l'exécution d'un tour de boucle préserve la vérité de P (Conservation) ; 3) qu'à la sortie de la boucle, P combinée à la négation de la condition de continuation démontre logiquement que le résultat calculé est exactement celui exigé par la spécification.

• Invariant de Boucle : Assertion ou propriété logique qui demeure VRAIE avant le début de la boucle et après chaque itération.

• Étape d'Initialisation : Démonstration que la propriété est vraie dès le départ, avant le premier tour de boucle.

• Étape de Conservation (Hérédité) : Preuve que si la propriété est vraie au début du tour k, elle reste vraie à la fin du tour k après la modification des variables.

• Étape de Conclusion (Correction) : Combinaison de l'invariant maintenu et du fait que la condition de boucle est devenue fausse, prouvant que le résultat final est exact.

Cas concret enseignant : *L'analogie de la tournée du livreur de journaux : Au départ de sa tournée, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison servie, les deux quantités changent, mais leur somme reste rigoureusement la même tout au long de la journée (invariant). Quand le sac est vide à la fin de la journée (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, absolument. Une boucle POUR est une spécialisation d'une boucle TANT QUE. On peut toujours simuler un POUR en initialisant le compteur avant le TANT QUE, en mettant la condition de borne dans le TANT QUE, et en incrémentant manuellement le compteur tout à la fin du corps de la boucle. Cependant, si le nombre de tours est connu, la boucle POUR est fortement recommandée car elle rend le code plus lisible et évite les oublis d'incrémentation.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition ($i < N$), est-ce que les instructions s'exécutent si i est déjà égal à N dès le début ?
Réponse du professeur : Non, pas du tout. La boucle TANT QUE est une boucle à pré-condition : l'expression logique ($i < N$) est évaluée avant toute exécution. Si i est égal à N dès le départ, la condition est évaluée à FAUX immédiatement et le programme saute directement après le bloc de boucle sans jamais exécuter son contenu (0 itération).

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, calculer 2 + 4 + 6 = 12). Justifier le choix de la boucle, rédiger le code et formaliser le variant de boucle prouvant la terminaison.

Résolution pas à pas :
Étape 1 - Choix de la structure : La valeur N est fournie dès l'entrée du programme, le nombre d'itérations est donc

parfaitement connu d'avance. La boucle 'POUR' est la structure optimale. Étape 2 - Initialisation des variables : 'somme' est un accumulateur initialisé à 0, 'terme' représente le premier entier pair strict (2). Étape 3 - Écriture du pseudo-code : Algorithme SommePremiersPairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 - Preuve de terminaison : Si l'on traduit cette itération sous forme de boucle générale avec compteur i, le variant de boucle est V = (N - i + 1). À chaque tour, i augmente de 1, donc V diminue strictement de 1. V est un entier minoré par 0. Dès que V atteint 0 (lorsque i dépasse N), la boucle s'arrête immédiatement.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle : La valeur affichée est 12. L'algorithme se termine de façon prouvée en N itérations et fournit le résultat correct.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale en mémoire et à l'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE teste la condition d'arrêt AVANT d'exécuter le corps (exécutée de 0 à N fois), tandis que la boucle RÉPÉTER... JUSQU'À teste la condition APRÈS avoir exécuté le corps (exécutée au moins 1 à N fois).

Q : Quelles sont les deux conditions mathématiques obligatoires que doit remplir une grandeur pour constituer un variant de boucle valide ?

R : La grandeur doit être une valeur entière strictement décroissante à chaque itération ($V_{k+1} < V_k$) et elle doit être minorée par une constante plancher (généralement 0) en dessous de laquelle la boucle s'arrête.

Q : Quel oubli classique dans la rédaction d'une boucle TANT QUE provoque une boucle infinie et fait perdre le maximum de points en DS de R1.01 ?

R : L'oubli de la mise à jour (incrémentation, modification ou re-saisie) de la variable de contrôle à l'intérieur du corps de la boucle, rendant la condition de sortie toujours vraie.